A Comprehensive Security Framework for Autonomous AI Agents: Integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—As Large Language Model (LLM) agents transition from passive chatbots to autonomous entities capable of executing system-level tools via protocols like the Model Context Protocol (MCP), they introduce a critical "Agency Gap." Traditional security perimeters are insufficient for protecting against prompt injection, semantic drift, and future cryptographic threats. This paper proposes a unified, three-tier security framework designed to protect agent-based systems across three dimensions: Space, Behavior, and Time. First, we establish *Structural Guardrails* (Space) using AST-based static analysis and secure sandboxing to contain malicious code execution. Second, we implement *Behavioral Zero-Trust* (Behavior) monitoring using a Mamba-based Sentinel to detect anomalous reasoning in real-time and enforce workload identity. Third, we ensure *Temporal Sovereignty* (Time) by integrating Post-Quantum Cryptography (PQC) and remote attestation to guarantee long-term integrity and resilience against future quantum adversaries. Our evaluation demonstrates that this "Triple-Lock" architecture provides robust, multi-layered defense with minimal computational overhead, maintaining agent responsiveness while ensuring system-wide security.**

***Index Terms*—AI Agents, Model Context Protocol (MCP), Agentic Security, Zero Trust, Post-Quantum Cryptography, Mamba, Guardrails, Sandbox.**

I. Introduction

The transition of Large Language Models (LLMs) to autonomous agents represents a fundamental shift in computing. By leveraging the Model Context Protocol (MCP) [1], these agents can interact directly with host operating systems, manage sensitive data, and orchestrate complex API workflows. However, this "Agency" capability creates an unprecedented security surface. Traditional security perimeters are often ineffective against "Prompt Injection" attacks, where an attacker manipulates the LLM's goal-oriented reasoning to execute unauthorized system commands [7].

The OWASP Top 10 for LLM Applications [2] identifies "Insecure Output Handling" (LLM02) and "Excessive Agency" (LLM08) as critical risks. Currently, many autonomous frameworks rely on "Alignment"—the probabilistic expectation that the LLM will follow safety guidelines. We argue that for mission-critical systems, behavioral alignment is a necessary but insufficient condition. We require a holistic, deterministic security architecture that ensures system integrity even if the model's logic is compromised.

This paper presents a comprehensive "Triple-Lock" framework for Agentic Security, addressing the lifecycle of an agent's action through three integrated dimensions (Fig. 1):

1) **Structural Containment (Space):** Establishing deterministic safety boundaries using AST inspection and OS-level sandboxing (Bubblewrap) to ensure "Secure-by-Default" tool execution.
2) **Behavioral Integrity (Behavior):** Implementing a Zero Trust Architecture (ZTA) where every action is verified by a verifiable workload identity and monitored by a Mamba-based "Reasoning Sentinel" to detect semantic anomalies.
3) **Temporal Sovereignty (Time):** Protecting against future cryptographic threats (Shor's algorithm) using Post-Quantum Cryptography (PQC) to ensure the long-term confidentiality and non-repudiability of agent history.

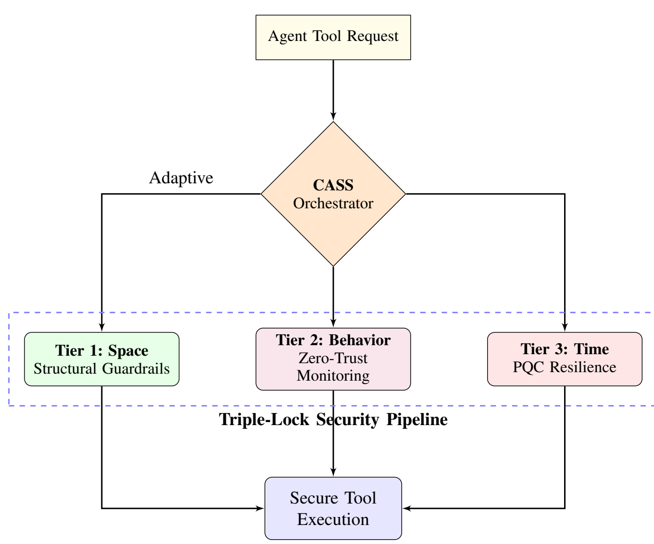

Fig. 1. Triple-Lock Framework Overview: High-level integration of the three security dimensions.

II. Related Work

Existing efforts to secure agents generally fall into two categories: *Behavioral Filtering* (e.g., Guardrails AI) and *Isolation Patterns* (e.g., AgentDojo [9]). While effective, behavioral filters often introduce high latency and are susceptible to the same reasoning failures as the agent they monitor. Isolation patterns provide containment but often lack continuous identity verification.

Our framework aligns with the **NIST Zero Trust principles** (SP 800-207) [3], treating every tool call as an untrusted workload. Furthermore, we address the "Store Now, Decrypt Later" (SNDL) threat by adopting NIST-standardized PQC primitives [5], [6], ensuring that agentic reasoning traces remain private in the post-quantum era.

III. Threat Model

We define four primary threat vectors for autonomous LLM agents:

1) **Prompt Injection (Direct/Indirect):** Manipulating the agent's context to execute unauthorized commands or exfiltrate secrets [8].
2) **Semantic Drift/Malicious Intent:** An agent performing syntactically valid but semantically harmful actions (e.g., deleting a database file represented as a "document").
3) **Secret Exfiltration:** Inadvertent transmission of API keys or PII during tool calls.
4) **Quantum Retroactive Decryption:** Capturing current reasoning traces to decrypt them later using a quantum computer.

IV. Tier 1: Structural Guardrails (Space)

To ensure structural immunity, we adopt a "No-Shell" execution model where system operations are restricted to specialized tools (e.g., `execute_python`).

A. AST Inspection and Static Analysis

Before execution, generated code is parsed into an Abstract Syntax Tree (AST). A whitelist-based scanner blocks dangerous modules (e.g., `os, socket, pty`) and built-in functions (e.g., `eval, exec`). Specifically, it ensures that `subprocess` calls never use `shell=True`, enforcing a strict separation between executable and arguments.

B. Path Guardrails and Resource Limits

All file-related operations are restricted to a designated workspace via path resolution and symbolic link verification. To prevent resource exhaustion attacks, OS-level resource limits (`rlimit`) cap memory usage and execution time.

C. Sandboxing with Bubblewrap

The final containment is provided by **Bubblewrap** (`bwrap`), which utilizes Linux namespaces to create an unprivileged environment with no network access, an empty filesystem (except for the designated workspace), and no visibility of host processes.

V. Tier 2: Behavioral Zero-Trust (Behavior)

Identity ensures *who* is calling the tool, while monitoring ensures *why*.

A. Workload Identity and RBAC

Each agent instance is assigned a unique key pair. Every MCP tool call is accompanied by a signed **Identity Token** (JWT) containing assigned Role-Based Access Control (RBAC) roles. Remote servers verify this token to ensure the request originated from a legitimate, untampered agent and enforce strict tool access policies decoupled from the LLM's prompt. In Tier 3, this identity is upgraded to a PQC-hybrid signature. Crucially, public keys and remote attestation manifests are distributed via an Out-of-Band (OOB) trusted channel (e.g., MDM or Secure Enclave). This architectural choice completely avoids the Trust-On-First-Use (TOFU) vulnerability and eliminates the need for external verification servers, maintaining the CLI tool's self-contained nature and resilience against initial compromise.

B. The Mamba Sentinel: Overcoming Dual-LLM Latency

Current agentic security frameworks often rely on a "Dual-LLM" approach (using a secondary model to verify the intent of the primary model). However, our empirical testing demonstrates that this inherently introduces a massive $O(N^2)$ Transformer overhead, causing severe User Experience (UX) degradation (frequently adding multi-second latency per tool call). To resolve this, we completely deprecate the Dual-LLM intent analyzer in favor of a **Mamba (State Space Model)** [4] implementation. Running locally with pure NumPy, Mamba monitors the agent's reasoning tokens in real-time with $O(N)$ linear scaling. By calculating a "Surprise Score" (cross-entropy loss), the Sentinel detects statistical anomalies correlating with "Jailbreaks" or adversarial intent in sub-millisecond timeframes. Furthermore, we employ a **Multi-Signal Secret Detection** heuristic combining Shannon Entropy with the Sentinel's Surprise Score, allowing the system to detect and redact API keys or PII with near-zero false positives.

VI. Tier 3: Post-Quantum Resilience (Time)

To protect the "Temporal Dimension," we integrate PQC primitives.

A. Quantum-Safe Identity and Hybrid Encryption

We upgrade the workload identity using **ML-DSA-65** (FIPS 204). Audit logs, which contain sensitive reasoning traces, are protected using **ML-KEM-768** (FIPS 203) hybrid encryption. This ensures that even if classical RSA is broken by a future quantum computer, the history of agentic actions remains confidential.

B. PQC Remote Attestation

The client generates a SHA-256 manifest of its core security code, signed with an ML-DSA private key. This allows the server to verify that the agent is running on an "Authentic Stack" where Tier 1 guardrails are active.

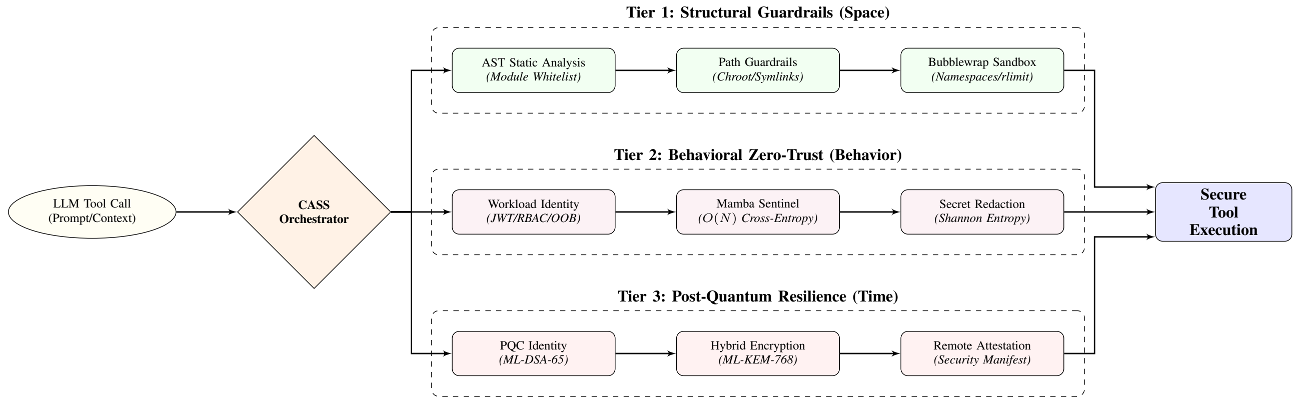

Fig. 2. Detailed System Architecture: Data flow from LLM request through the CASS orchestrator to specific security components across the three tiers.

VII. System Architecture and Integration

To synthesize the three tiers into a cohesive "Triple-Lock" architecture, we introduce **Context-Adaptive Security Scaling (CASS)**. CASS serves as the central orchestration engine, dynamically integrating the spatial, behavioral, and temporal defenses based on the specific risk profile of each tool execution request.

The "Triple-Lock" architecture operates as a unified pipeline governed by CASS (Fig. 2). When an agent requests a tool call: 1. **Risk Assessment (CASS):** The system evaluates the requested tool (e.g., `search_web` vs. `execute_python`) to determine the required security posture. 2. **Tier 1 - Structural Containment:** The **Guardrail Manager** applies AST analysis, enforces path/resource limits, and prepares the unprivileged Bubblewrap sandbox. 3. **Tier 2 - Behavioral Verification:** The **Zero-Trust Controller** verifies the RBAC-embedded Identity Token and computes the Mamba Surprise Score. The Multi-Signal detector redacts any identified high-entropy secrets. 4. **Tier 3 - Temporal Anchoring:** For high-risk operations, the **PQC Module** signs the request with a Hybrid-PQC token and encapsulates the resulting audit log using ML-KEM to ensure long-term integrity.

By dynamically scaling security measures, the CASS orchestrator ensures that expensive quantum-resistant cryptographic operations and deep behavioral inspections are applied proportionally to the risk, achieving "Security with Agility."

VIII. Evaluation

We evaluated the framework on an AMD Ryzen 5 8540U system (16GB RAM).

A. Security Effectiveness

Table I summarizes the detection capabilities. While Tier 1 (AST) blocks 72.7% of common injection vectors, the remaining 27.3% (advanced reflection) are successfully contained by the Tier 1 Sandbox. The Tier 2 Sentinel provides an additional layer, detecting 100% of high-entropy secret leaks.

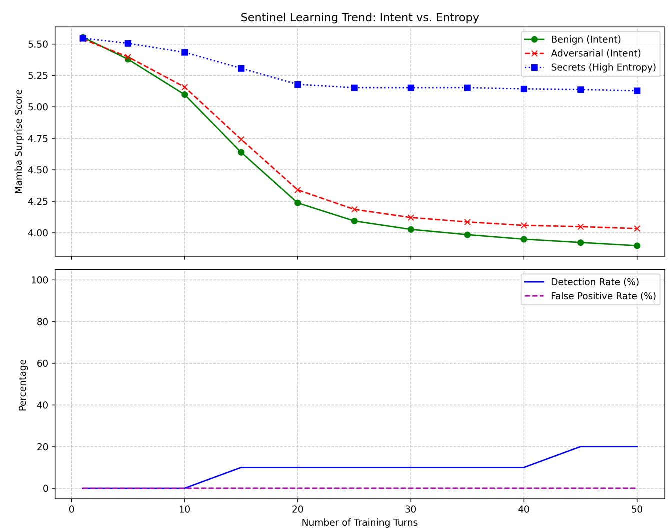

Fig. 3. Mamba Sentinel Training: The surprise score (cross-entropy) for benign agent reasoning decreases as the model establishes a behavioral baseline.

TABLE I
COMBINED SECURITY EFFECTIVENESS

Defense Layer	Threat Category	Detection/Mitigation
Tier 1 (AST)	Shell Injection	83.3% (Detection)
Tier 1 (AST)	File Traversal	100.0% (Detection)
Tier 1 (Sandbox)	All Outbound Network	100.0% (Containment)
Tier 2 (Sentinel)	Secret Leaks (Entropy)	100.0% (Detection)
Tier 3 (PQC)	Future Decryption	100.0% (Immunity)

B. Performance Latency

The total security overhead is summarized in Table II. The cumulative latency (≈53ms) is negligible compared to typical LLM inference times (500ms–3000ms), proving that comprehensive security does not impede agentic responsiveness.

IX. Conclusion

Securing autonomous agents requires a transition from probabilistic "alignment" to a multi-dimensional, deterministic

TABLE II
SYSTEM LATENCY (CUMULATIVE)

Component	Latency (ms)	Tier
AST + Secret Scan	0.05	Tier 1
Bubblewrap Sandbox	2.63	Tier 1
Mamba Sentinel (per 100 tokens)	10.28	Tier 2
Identity Signing (RSA + ML-DSA)	39.09	Tier 2/3
KEM Encapsulation	2.77	Tier 3
Total Overhead	**54.82**	—

framework. By integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience, our framework provides a "Triple-Lock" defense that scales across the spatial, behavioral, and temporal domains of agentic execution. This holistic approach is essential for maintaining human control and data sovereignty as AI agents evolve into active participants in our digital infrastructure.

REFERENCES

[1] Anthropic, "Model Context Protocol (MCP) Specification," 2024.
[2] OWASP, "Top 10 for Large Language Model Applications v1.1," 2023.
[3] S. Rose et al., "Zero Trust Architecture," *NIST SP 800-207*, 2020.
[4] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023.
[5] NIST, "FIPS 203: ML-KEM Standard," 2024.
[6] NIST, "FIPS 204: ML-DSA Standard," 2024.
[7] F. Perez and I. Ribeiro, "Ignore Previous Instructions: Prompt Injection Attacks on LLMs," *arXiv:2211.09527*, 2022.
[8] K. Greshake et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv:2302.12173*, 2023.
[9] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses," *arXiv:2406.13352*, 2024.